

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler

# Load dataset
df = pd.read_csv("/content/AirPassengers.csv")

print(df.head())
print(df.shape)

# Convert date column
df['Month'] = pd.to_datetime(df['Month'])

# Get passenger values
data = df['#Passengers'].values.reshape(-1, 1)

# Normalize data between 0 and 1
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)

# Create sequences
sequence_length = 12

X = []
y = []

for i in range(len(data_scaled) - sequence_length):
    X.append(data_scaled[i:i + sequence_length])
    y.append(data_scaled[i + sequence_length])

X = np.array(X)
y = np.array(y)

print("X shape:", X.shape)
print("y shape:", y.shape)

```

```

      Month  #Passengers
0  1949-01         112
1  1949-02         118
2  1949-03         132
3  1949-04         129
4  1949-05         121
(144, 2)
X shape: (132, 12, 1)
y shape: (132, 1)

```

```

import torch
import torch.nn as nn

# Convert NumPy arrays to PyTorch tensors
X_tensor = torch.tensor(X, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.float32)

# Define RNN model
class RNNModel(nn.Module):
    def __init__(self):
        super(RNNModel, self).__init__()

        self.rnn = nn.RNN(
            input_size=1,
            hidden_size=50,
            num_layers=1,
            batch_first=True
        )

        self.fc = nn.Linear(50, 1)

    def forward(self, x):
        out, _ = self.rnn(x)

        # Take output from last time step
        out = out[:, -1, :]

        out = self.fc(out)

        return out

model = RNNModel()

```

```
print(model)
```

```
RNNModel(
  (rnn): RNN(1, 50, batch_first=True)
  (fc): Linear(in_features=50, out_features=1, bias=True)
)
```

```
# Loss function
criterion = nn.MSELoss()

# Optimizer
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

epochs = 200
losses = []

for epoch in range(epochs):

    # Forward pass
    output = model(X_tensor)

    # Calculate loss
    loss = criterion(output, y_tensor)

    # Backpropagation
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    losses.append(loss.item())

    if (epoch + 1) % 20 == 0:
        print(
            f"Epoch [{epoch+1}/{epochs}], "
            f"Loss: {loss.item():.6f}"
        )

print("Training completed.")
```

```
Epoch [20/200], Loss: 0.036389
Epoch [40/200], Loss: 0.020689
Epoch [60/200], Loss: 0.008704
Epoch [80/200], Loss: 0.007894
Epoch [100/200], Loss: 0.007013
Epoch [120/200], Loss: 0.006291
Epoch [140/200], Loss: 0.005658
Epoch [160/200], Loss: 0.005132
Epoch [180/200], Loss: 0.004625
Epoch [200/200], Loss: 0.004074
Training completed.
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

# Make predictions
with torch.no_grad():
    predictions = model(X_tensor).numpy()

# Convert normalized values back to original passenger numbers
predictions_original = scaler.inverse_transform(predictions)
y_original = scaler.inverse_transform(y)

# Calculate metrics
mse = mean_squared_error(y_original, predictions_original)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_original, predictions_original)

print("MSE :", mse)
print("RMSE:", rmse)
print("MAE :", mae)
```

```
MSE : 1085.2947300896335
RMSE: 32.943811711604255
MAE : 24.080238168889824
```

```
# Plot training loss
plt.figure(figsize=(10, 5))

plt.plot(losses)

plt.title("RNN Training Loss")
plt.xlabel("Epoch")
plt.ylabel("MSE Loss")
```

```
plt.show()

# Plot actual vs predicted passengers
plt.figure(figsize=(12, 5))

plt.plot(y_original, label="Actual Passengers")
plt.plot(predictions_original, label="Predicted Passengers")

plt.title("Airline Passenger Prediction using RNN")
plt.xlabel("Time")
plt.ylabel("Number of Passengers")

plt.legend()
plt.show()
```



